Software Requirements Specification

for

SWE Autopilot

AI-Driven Autonomous Software Engineering Platform with Multi-Agent Development and Intelligent Requirement Change Management

Version 1.0 — Draft for Advisor Review

Prepared by

Abdul Wasay (Registration No. 2380101)
Ahmed Hassan Sheikh (Registration No. 2380102)

Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology University (SZABIST), Karachi Campus

Supervisor: Adil Ahmed Unar

Date created: October 21, 2026

Copyright © 1999 by Karl E. Wiegers. Template adapted for academic project documentation.

Table of Contents

Revision History **4**

1. Introduction **5**

1.1 Purpose . . . 5

1.2 Document Conventions . . . 5

1.3 Intended Audience and Reading Suggestions . . . 5

1.4 Product Scope . . . 5

1.5 References . . . 5

2. Overall Description **6**

2.1 Product Perspective . . . 6

2.2 Product Functions . . . 6

2.3 User Classes and Characteristics . . . 6

2.4 Operating Environment . . . 6

2.5 Design and Implementation Constraints . . . 7

2.6 User Documentation . . . 7

2.7 Assumptions and Dependencies . . . 7

3. External Interface Requirements **7**

3.1 User Interfaces . . . 7

3.2 Hardware Interfaces . . . 7

3.3 Software Interfaces . . . 8

3.4 Communications Interfaces . . . 8

4. System Features **8**

4.1 User Registration and Authentication . . . 8

4.1.1 Feature Summary . . . 8

4.1.2 Functional Requirements . . . 9

4.2 Requirement Analysis and Ambiguity Detection . . . 9

4.2.1 Feature Summary . . . 9

4.2.2 Functional Requirements . . . 10

4.3 Requirement-to-Artifact Traceability . . . 10

4.3.1 Feature Summary . . . 10

4.3.2 Functional Requirements . . . 11

4.4 Predictive Risk Analysis . . . 11

4.4.1 Feature Summary . . . 11

4.4.2 Functional Requirements . . . 11
4.5 Architect/Critic Review and Approval . . . 12
4.5.1 Feature Summary . . . 12
4.5.2 Functional Requirements . . . 12
4.6 Multi-Agent Code Generation . . . 12
4.6.1 Feature Summary . . . 12
4.6.2 Functional Requirements . . . 13
4.7 Docker Sandbox Execution . . . 13
4.7.1 Feature Summary . . . 13
4.7.2 Functional Requirements . . . 13
4.8 Initial Evidence-Based Verification . . . 14
4.8.1 Feature Summary . . . 14
4.8.2 Functional Requirements . . . 14
4.9 Project Workspace and Artifact Management . . . 14
4.9.1 Feature Summary . . . 14
4.9.2 Functional Requirements . . . 15
5. Other Nonfunctional Requirements **15**
5.1 Performance Requirements . . . 15
5.2 Safety Requirements . . . 15
5.3 Security Requirements . . . 15
5.4 Software Quality Attributes . . . 16
5.5 Business Rules . . . 16
6. Other Requirements **16**
Appendix A: Glossary **17**
Appendix B: Analysis Models **17**
Appendix C: To Be Determined List **17**

Revision History

Name	Date	Reason For Changes	Version
Abdul Wasay / Ahmed Hassan Sheikh	October 21, 2026	Initial FYP-I baseline prepared for advisor review	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the FYP-I requirements for SWE Autopilot, an AI-driven autonomous software engineering platform with a controlled multi-agent workflow. It covers the requirement analyzer, traceability, predictive risk analysis, Agent Runtime foundations, Architect/Critic review, code generation, Docker sandbox execution, initial verification, workspace management, and supporting authentication. FYP-II functions are explicitly outside this baseline.

1.2 Document Conventions

The document uses Times New Roman, 12-point body text, 1.2 line spacing, normal margins, and justified paragraphs. Main headings use 16-point bold text and subsection headings use 14-point bold text. Functional requirements use unique identifiers. Priority values are High, Medium, or Low. Status values are Pending until the advisor reviews and approves the baseline. The words shall and must identify mandatory requirements.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the project advisor, project team, developers, testers, and reviewers. Advisors should review Sections 1, 2, 4, and 5 for scope and acceptance criteria. Developers should use Sections 3 and 4 for interfaces and behavior. Testers should derive tests from the functional and nonfunctional requirements.

1.4 Product Scope

SWE Autopilot provides a web-based control plane that transforms natural-language software requirements into structured, traceable, and verifiable engineering work. A deterministic Agent Runtime controls LLM-based reasoning, tool permissions, artifact versions, execution evidence, and approval decisions. The FYP-I release demonstrates the foundation of this workflow for controlled full-stack web application development.

• Analyze natural-language requirements and detect ambiguity, incompleteness, conflicts, and missing acceptance criteria.

• Create requirement IDs, user stories, acceptance criteria, and artifact relationships.

• Predict architectural, technical, performance, security, dependency, and implementation risks.

• Generate and critique architecture alternatives before implementation.

• Generate frontend, backend, API, database, configuration, test, and documentation artifacts through controlled agents.

• Execute generated software inside Docker and collect build/runtime evidence.

• Run the initial verification workflow and associate results with artifacts.

1.5 References

1. SWE Autopilot FYP Proposal, Complete Unified Document, October 7, 2026.

2. Software Requirements Specification Template (2025), supplied by the project advisor.

3. Sign Up Use Case Example, supplied by the project advisor.

4. SWE Autopilot repository: https://github.com/sheikh4467/SWE-Autopilot

2. Overall Description

2.1 Product Perspective

SWE Autopilot is a new self-contained web platform composed of a Next.js frontend, FastAPI backend, Agent Runtime, specialized SDLC agents, data storage, and controlled Docker execution. The frontend provides the landing/authentication experience, project workspace, requirements input, traceability view, risk review, architecture review, code-generation controls, and verification status. The target architecture uses PostgreSQL, Redis, LangGraph/LangChain, LLM providers, and Docker.

2.2 Product Functions

• Register and authenticate project members using email/password and configured Google/GitHub OAuth.

• Create a project workspace and submit natural-language requirements.

• Compile requirements into structured artifacts and detect ambiguity.

• Maintain requirement-to-artifact traceability.

• Analyze and prioritize predictive risks.

• Coordinate Architect/Critic proposals and approval.

• Generate code with specialized agents under explicit permissions.

• Build and run code in a Docker sandbox.

• Collect initial verification evidence and preserve execution history.

2.3 User Classes and Characteristics

User Class	**Characteristics and Responsibilities**
Project Member / Developer	Creates workspaces, submits requirements, reviews artifacts, approves architecture, and inspects evidence.
Advisor / Reviewer	Reviews scope, requirements, architecture decisions, evidence, and unresolved issues.
System Administrator	Configures providers, repository access, execution limits, and environment settings.
Agent / Runtime	Internal actors that analyze, propose, critique, generate, and coordinate under runtime controls.

2.4 Operating Environment

The system shall operate as a browser-accessible web application. Development may use Windows, Linux, or macOS with a multi-core processor and at least 16 GB RAM recommended. The target stack includes Python 3.11+, Node.js, Docker Desktop, PostgreSQL, Redis, Git, a modern browser, and an LLM API or local Ollama model.

2.5 Design and Implementation Constraints

• The Agent Runtime shall control workflow state, tools, permissions, artifact versions, and approvals.

• Generated code shall execute through a restricted Docker sandbox rather than unrestricted host execution.

• OAuth client secrets, session secrets, and model keys shall be loaded from protected environment variables.

• FYP-I targets full-stack web applications; native desktop, embedded, hardware, and native mobile compilers are excluded.

• The architecture shall remain model-provider independent.

2.6 User Documentation

• Quick-start guide

• Workspace and requirements guide

• Advisor review checklist

• Developer setup guide for OAuth, Docker, and model providers

2.7 Assumptions and Dependencies

• A modern browser and reachable deployment are available.

• Docker is available for sandbox execution.

• At least one LLM provider or local model is configured.

• The target project is stored in a Git repository.

• PostgreSQL and Redis are available for the target deployment.

3. External Interface Requirements

3.1 User Interfaces

The frontend shall provide a responsive landing/authentication page with 3D visuals, scroll transitions, and clear Sign In and Create Account actions. The authenticated workspace shall provide views for requirements, traceability, risks, architecture, code generation, sandbox execution, and verification evidence. Forms shall show validation and actionable errors. Tables shall show IDs, priorities, versions, relationships, approval states, execution results, and verification evidence.

3.2 Hardware Interfaces

No dedicated hardware interface is required. Docker may use CPU, memory, storage, and network resources subject to configured limits. Hardware-based development is outside FYP-I scope.

3.3 Software Interfaces

Interface	Purpose and Data Exchanged
Next.js ↔ FastAPI	REST/JSON authentication, projects, requirements, artifacts, agent runs, and evidence.
FastAPI ↔ PostgreSQL	Project, requirement, artifact, relationship, risk, architecture, execution, and evidence records.
Agent Runtime ↔ LangGraph/LangChain	State transitions, messages, tool requests, permissions, and outcomes.
Runtime ↔ LLM provider	Structured prompts and responses for analysis, architecture, critique, and generation.
Runtime ↔ Docker	Build/run commands, restricted mounts, logs, exit status, and runtime results.
Authentication ↔ Google/GitHub	Server-side OAuth authorization code, provider identity, email, and profile data.

3.4 Communications Interfaces

Browser-to-backend communication shall use HTTP during local development and HTTPS in deployment. REST requests shall use JSON except for OAuth redirects. OAuth state shall be validated server-side. Authenticated sessions shall use HTTP-only cookies. Provider, LLM, and deployment communications shall use TLS-protected endpoints.

4. System Features

This section defines the FYP-I system features and their functional requirements at SRS level. Detailed actors, preconditions, stimulus/response sequences, alternate flows, exception flows, and postconditions are maintained in the separate SWE Autopilot FYP-I System Use Case Specification. All statuses are Pending until the advisor reviews the document.

4.1 User Registration and Authentication

4.1.1 Feature Summary

Feature ID	FYP1-F1
Feature Name	User Registration and Authentication
Description	The platform allows a project member to create an account and access the SWE Autopilot workspace using email/password authentication or configured Google and GitHub OAuth.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.1.1. User Registration and Authentication Feature Summary

4.1.2 Functional Requirements

Req.ID	Description	Priority	Status
AUTH-01	The system shall display sign-up and sign-in forms with required name, email, and password fields as appropriate.	High	Pending
AUTH-02	The system shall validate email format, required fields, and a minimum password length of eight characters before submission.	High	Pending
AUTH-03	The system shall prevent duplicate local accounts for the same normalized email address.	High	Pending
AUTH-04	The system shall hash passwords using a one-way password hashing function and shall never store plaintext passwords.	High	Pending
AUTH-05	The system shall support configured Google and GitHub OAuth login through server-side callback routes.	High	Pending
AUTH-06	The system shall create an HTTP-only authenticated session containing only the application user identifier.	High	Pending
AUTH-07	The system shall display actionable errors for invalid credentials, provider failures, and unavailable services.	High	Pending

Table 4.1.2. User Registration and Authentication Functional Requirements

4.2 Requirement Analysis and Ambiguity Detection

4.2.1 Feature Summary

Feature ID	FYP1-F2
Feature Name	Requirement Analysis and Ambiguity Detection
Description	The Requirement Analyzer converts a natural-language software requirement into structured requirement artifacts and identifies ambiguity, incompleteness, conflicts, and missing acceptance criteria.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.2.1. Requirement Analysis and Ambiguity Detection Feature Summary

4.2.2 Functional Requirements

Req.ID	Description	Priority	Status
REQ-AN-01	The system shall accept a natural-language requirement for a selected project.	High	Pending
REQ-AN-02	The system shall create unique IDs for extracted requirements, user stories, and acceptance criteria.	High	Pending
REQ-AN-03	The system shall classify extracted items as functional or nonfunctional requirements where applicable.	High	Pending
REQ-AN-04	The system shall identify ambiguous, incomplete, conflicting, or underspecified requirement statements.	High	Pending
REQ-AN-05	The system shall explain each ambiguity or missing information finding in user-readable language.	High	Pending
REQ-AN-06	The system shall allow the user to review, edit, accept, or reject generated requirement artifacts.	High	Pending
REQ-AN-07	The system shall preserve the original requirement text and the version history of analysis results.	High	Pending

Table 4.2.2. Requirement Analysis and Ambiguity Detection Functional Requirements

4.3 Requirement-to-Artifact Traceability

4.3.1 Feature Summary

Feature ID	FYP1-F3
Feature Name	Requirement-to-Artifact Traceability
Description	The Traceability Manager links requirements to user stories, architecture components, APIs, code artifacts, tests, documentation, and verification evidence.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.3.1. Requirement-to-Artifact Traceability Feature Summary

4.3.2 Functional Requirements

Req.ID	Description	Priority	Status
TRACE-01	The system shall display requirement-to-artifact relationships for the selected project.	High	Pending
TRACE-02	The system shall support relationships between requirements, user stories, architecture, APIs/components, code, tests, documentation, and evidence.	High	Pending
TRACE-03	The system shall preserve stable IDs and version references for linked artifacts.	High	Pending
TRACE-04	The system shall identify missing or unresolved links in the traceability chain.	High	Pending
TRACE-05	The system shall allow an authorized user to add, edit, or remove a relationship.	High	Pending
TRACE-06	The system shall record relationship changes in the project audit history.	High	Pending

Table 4.3.2. Requirement-to-Artifact Traceability Functional Requirements

4.4 Predictive Risk Analysis

4.4.1 Feature Summary

Feature ID	FYP1-F4
Feature Name	Predictive Risk Analysis
Description	The Predictive Risk Analyzer evaluates requirements and proposed architecture for technical, architectural, performance, security, dependency, ambiguity, and implementation-complexity risks.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.4.1. Predictive Risk Analysis Feature Summary

4.4.2 Functional Requirements

Req.ID	Description	Priority	Status
RISK-01	The system shall run a risk analysis for a selected project baseline.	High	Pending
RISK-02	The system shall evaluate at least architecture, technical, performance, security, dependency, ambiguity, and implementation-complexity risk categories.	High	Pending
RISK-03	The system shall assign a severity or priority to each identified risk.	High	Pending
RISK-04	The system shall identify the requirement or artifact affected by each risk where possible.	High	Pending
RISK-05	The system shall provide a rationale and mitigation suggestion for each risk.	High	Pending
RISK-06	The system shall allow an authorized user to acknowledge, annotate, or defer a risk with a recorded rationale.	High	Pending

Table 4.4.2. Predictive Risk Analysis Functional Requirements

4.5 Architect/Critic Review and Approval

4.5.1 Feature Summary

Feature ID	FYP1-F5
Feature Name	Architect/Critic Review and Approval
Description	The Architect Agent proposes a system architecture and the Critic Agent evaluates it for security, scalability, complexity, missing components, dependency problems, and requirement mismatches.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.5.1. Architect/Critic Review and Approval Feature Summary

4.5.2 Functional Requirements

Req.ID	Description	Priority	Status
ARCH-01	The system shall generate at least one architecture proposal from the reviewed project requirements.	High	Pending
ARCH-02	The system shall represent components, APIs, data stores, technology choices, and communication patterns in a structured proposal.	High	Pending
ARCH-03	The Critic Agent shall evaluate security, scalability, complexity, missing components, dependencies, and requirement coverage.	High	Pending
ARCH-04	The system shall allow comparison of architecture alternatives and record trade-offs.	High	Pending
ARCH-05	The system shall store an architecture decision record with rationale and review history.	High	Pending
ARCH-06	The system shall require an approval state before an architecture proposal can be used for controlled code generation.	High	Pending

Table 4.5.2. Architect/Critic Review and Approval Functional Requirements

4.6 Multi-Agent Code Generation

4.6.1 Feature Summary

Feature ID	FYP1-F6
Feature Name	Multi-Agent Code Generation
Description	The platform uses specialized agents to generate frontend, backend, API, database, configuration, test, and documentation artifacts while the Agent Runtime controls tools, permissions, repositories, and versions.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.6.1. Multi-Agent Code Generation Feature Summary

4.6.2 Functional Requirements

Req.ID	Description	Priority	Status
CODE-01	The system shall create a generation plan from approved requirements and architecture artifacts.	High	Pending
CODE-02	The system shall support specialized agents for frontend, backend/API, database/configuration, tests, and documentation artifacts.	High	Pending
CODE-03	The Agent Runtime shall enforce tool, repository, and artifact permissions for each agent.	High	Pending
CODE-04	The system shall preserve generated files, diffs, dependencies, prompts or references, and execution status as a versioned artifact set.	High	Pending
CODE-05	The system shall allow a user to review and approve or reject a generated change set before sandbox execution.	High	Pending
CODE-06	The system shall not label generated code as verified until it passes the verification workflow.	High	Pending

Table 4.6.2. Multi-Agent Code Generation Functional Requirements

4.7 Docker Sandbox Execution

4.7.1 Feature Summary

Feature ID	FYP1-F7
Feature Name	Docker Sandbox Execution
Description	The Docker Sandbox builds and runs generated applications in a controlled environment and collects process output, exit status, build results, and startup results.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.7.1. Docker Sandbox Execution Feature Summary

4.7.2 Functional Requirements

Req.ID	Description	Priority	Status
SBOX-01	The system shall execute approved generated artifacts inside a Docker sandbox.	High	Pending
SBOX-02	The sandbox shall apply configured restrictions for permissions, time, memory, storage, and process behavior where supported.	High	Pending
SBOX-03	The system shall capture build output, application startup output, standard output, standard error, exit code, and duration.	High	Pending
SBOX-04	The system shall prevent direct host execution when a sandbox execution is required.	High	Pending
SBOX-05	The system shall record timeout, resource-limit, policy-block, and infrastructure-failure events.	High	Pending
SBOX-06	The system shall associate each sandbox run with the exact generated artifact version and project.	High	Pending

Table 4.7.2. Docker Sandbox Execution Functional Requirements

4.8 Initial Evidence-Based Verification

4.8.1 Feature Summary

Feature ID	FYP1-F8
Feature Name	Initial Evidence-Based Verification
Description	The Initial Verification System evaluates generated software using available build, runtime, test, API, static-analysis, schema, and health evidence and reports whether the FYP-I verification conditions are satisfied.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.8.1. Initial Evidence-Based Verification Feature Summary

4.8.2 Functional Requirements

Req.ID	Description	Priority	Status
VER-01	The system shall list configured verification checks for a selected generated artifact version.	High	Pending
VER-02	The system shall execute available build, runtime, Pytest, Playwright, API, static-analysis, schema, and health checks as configured.	High	Pending
VER-03	The system shall capture the command, output, duration, exit status, and environment reference for each check.	High	Pending
VER-04	The system shall classify each check as passed, failed, skipped, or unavailable with a reason.	High	Pending
VER-05	The system shall associate verification evidence with the generated artifact and relevant traceability links.	High	Pending
VER-06	The system shall prevent an incomplete or failed verification run from being represented as a successful verification.	High	Pending

Table 4.8.2. Initial Evidence-Based Verification Functional Requirements

4.9 Project Workspace and Artifact Management

4.9.1 Feature Summary

Feature ID	FYP1-F9
Feature Name	Project Workspace and Artifact Management
Description	The platform provides a workspace for projects, artifacts, agent runs, approvals, versions, execution records, and verification evidence so FYP-I work remains organized and auditable.
Priority	High
Scope	FYP-I baseline; detailed flows are documented in the separate System Use Case Specification.

Table 4.9.1. Project Workspace and Artifact Management Feature Summary

4.9.2 Functional Requirements

Req.ID	Description	Priority	Status
WORK-01	The system shall allow an authenticated project member to create and open a project workspace.	High	Pending
WORK-02	The system shall display project status, recent agent runs, risks, approvals, unresolved issues, and verification state.	High	Pending
WORK-03	The system shall store and retrieve versioned requirements, user stories, architecture, code, tests, documentation, and evidence artifacts.	High	Pending
WORK-04	The system shall associate artifacts with their creator, timestamp, version, and source workflow action.	High	Pending
WORK-05	The system shall enforce project-level access control for workspace data and actions.	High	Pending
WORK-06	The system shall preserve an audit history for approvals, permission decisions, execution runs, and artifact changes.	High	Pending

Table 4.9.2. Project Workspace and Artifact Management Functional Requirements

5. Other Nonfunctional Requirements

5.1 Performance Requirements

• The landing page shall become interactive within 3 seconds in the target local environment.

• The backend shall return validation errors within 2 seconds under normal local load.

• A requirement analysis request shall show a running state within 3 seconds.

• Workspace views shall load the latest saved state within 3 seconds for the FYP-I target dataset.

• Docker execution shall stream or persist logs for inspection.

5.2 Safety Requirements

• Generated code shall not run directly on the host when sandbox execution is required.

• The sandbox shall apply time, memory, storage, and process limits where supported.

• The runtime shall stop an execution that exceeds policy limits.

• Architecture and generated artifacts shall require explicit review before promotion.

5.3 Security Requirements

• Passwords shall be stored only as strong one-way hashes.

• OAuth state shall be generated, validated, and consumed server-side.

• Sessions shall use HTTP-only cookies and shall not expose provider secrets in URLs.

• OAuth and LLM secrets shall be supplied through protected environment variables.

• Agent tools and artifacts shall be permission controlled and audited.

• Project data shall be isolated by user and workspace authorization.

5.4 Software Quality Attributes

Attribute	Requirement
Usability	A first-time project member should be able to create a workspace, submit a requirement, and find the first analysis result without external assistance.
Correctness	An artifact shall not be marked verified without recorded execution or validation evidence.
Reliability	Failed runs shall produce visible failure states and diagnostics.
Maintainability	Frontend, backend, runtime, agents, and storage adapters shall be separable modules.
Portability	The platform shall run in local Windows/Linux/macOS and container-capable deployment environments.
Testability	Requirements shall have identifiable inputs, outputs, error paths, and evidence.
Traceability	Requirements and artifacts shall carry stable IDs and relationships.

5.5 Business Rules

• A project member may initiate analysis only for an authorized workspace.

• An LLM response alone shall not mark an artifact verified.

• Architecture decisions require a rationale and approval before controlled generation.

• Generated code must remain linked to the requirements and architecture that motivated it.

• FYP-II capabilities shall not be represented as completed FYP-I functionality.

6. Other Requirements

• The system shall preserve timestamps and version identifiers for requirements, artifacts, runs, approvals, and evidence.

• The system shall support export of a project summary.

• The system shall support UTF-8 natural-language requirements and generated documentation.

• The system shall keep FYP-I and FYP-II scope labels visible in planning and review views.

Appendix A: Glossary

Term	Definition
Agent Runtime	Deterministic control layer for workflow state, permissions, artifacts, evidence, approvals, and audit history.
Traceability	Relationships connecting requirements to downstream engineering artifacts and evidence.
Docker Sandbox	Restricted environment for building and running generated software.
Verification Evidence	Observable build, runtime, test, static-analysis, schema, or execution output.
FYP-I	First semester scope covering requirements, runtime foundations, architecture, generation, sandboxing, and initial verification.
LLM	Large Language Model used for reasoning or generation inside the controlled workflow.

Appendix B: Analysis Models

Top-level FYP-I data flow: Project Member → Requirement Input → Requirement Compiler → Structured Requirements / User Stories / Acceptance Criteria → Traceability Store → Risk Analyzer → Architect/Critic → Approved Architecture → Code Agents → Docker Sandbox → Verification Evidence → Project Workspace.

Primary traceability path: REQ-### → US-### → ARCH-### → API/COMPONENT → CODE ARTIFACT → TEST ARTIFACT → EXECUTION RUN → VERIFICATION EVIDENCE.

FYP-I state flow: Draft Requirement → Analyzed → Reviewed → Architecture Proposed → Architecture Approved → Code Generated → Sandbox Executed → Verification Passed / Failed / Escalated.

The complete logical ERD is intentionally maintained as a separate stand-alone document: SWE_Autopilot_Whole_Project_ERD_CrowFoot.pdf. It is not embedded in this SRS.

Appendix C: To Be Determined List

TBD ID	Open Item	Owner / Resolution Point
TBD-01	Final PostgreSQL schema and migration strategy.	Team; platform-core design.
TBD-02	Final LLM model/provider for the advisor demonstration.	Team and advisor; before evaluation.
TBD-03	Docker resource limits and network policy.	Team; before sandbox verification.
TBD-04	Advisor approval workflow and roles.	Advisor and team; review-gate design.
TBD-05	Benchmark dataset and acceptance thresholds.	Team; before FYP-I evaluation.